

**This chapter covers the following topics related to Objective 5.5 (Given a scenario, use the appropriate tool or protocol to solve networking issues) of the CompTIA Network+ N10-009 certification exam:**

- Software tools
  - Protocol analyzer
  - Command line
    - ping
    - traceroute/tracert
    - nslookup
    - tcpdump
    - dig
    - netstat
    - ip/ifconfig/ipconfig
    - arp
  - Nmap
  - Link Layer Discovery Protocol (LLDP)/Cisco Discovery Protocol (CDP)
  - Speed tester
- Hardware tools
  - Toner
  - Cable tester
  - Taps
  - Wi-Fi analyzer
  - Visual fault locator
- Basic networking device commands
  - show mac-address-table
  - show route
  - show interface
  - show config
  - show arp
  - show vlan
  - show power

CHAPTER 25

# Network Troubleshooting Tools

Troubleshooting your network and proactively stopping issues before they become trouble tickets are made much easier for you by a wide variety of software, hardware, and command-line tools that you can use with ease.

In this chapter, you will learn about a number of these tools that can make maintenance and troubleshooting of your network much easier. This chapter also reviews several commands that are very important on common network devices.

# Foundation Topics

## Software Tools

An array of software tools are designed to help with managing networks. This section provides an excellent starting point, introducing you to some of the commonly used tools, many of which are freely available.

### Protocol Analyzer/Packet Capture

If you understand the characteristics of the protocols running on your network (such as the fields in a protocol's header), a ***protocol analyzer*** (also known as a *network sniffer or* simply *packet capture* tool) can be a tremendous troubleshooting asset. A protocol analyzer can be a standalone device or software running on a laptop or other mobile computer. You can use a protocol analyzer to capture traffic flowing through a network switch, using the port mirroring feature of the switch. By examining the captured packets, you can discern the details of communication flows (sessions) as they are being set up, maintained, and torn down. The examination of these captured packets, referred to as *traffic analysis*, provides an administrator with valuable insights about the nature of traffic flowing through a network.

Protocol analyzers come with a wide range of features and at a variety of price points. Wireshark is a free software program that can make your laptop act like a protocol analyzer. Protocol analyzers can assist in identifying details such as top talkers, top destinations, top protocols in use, and quantity of traffic on the network. You can download a free copy of Wireshark from https://www.wireshark.org. Figure 25-1 shows the Wireshark protocol analyzer application.

### Bandwidth Speed Tester

Many bandwidth ***speed testers*** are available to assist you in verifying throughput from a local computer to an Internet site. One example is https://www.speedtest.net. Using sites such as this can assist you in determining whether an overall connection to the Internet is slow or whether just a specific site or server is slow to respond.

### Port Scanner

A *port scanner* is a useful software tool that probes a network device to determine which ports are open on the device. A port scanner can send port scans to a single device, or it can perform a port sweep, where it checks for open ports on a number of network devices at one time.

Key Topic

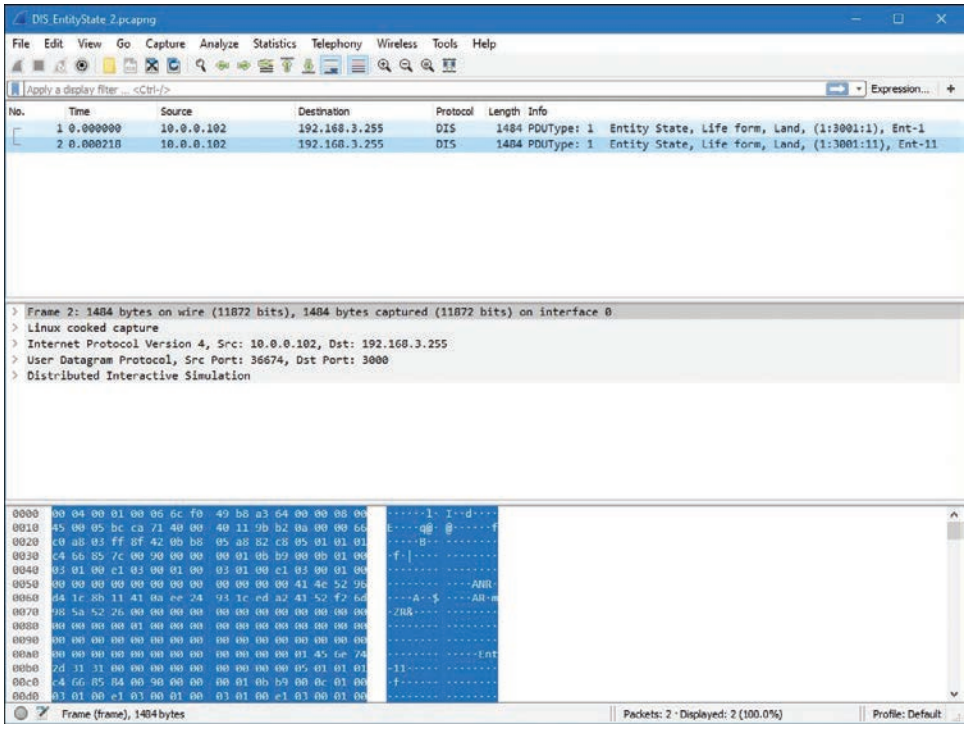



**Figure 25-1** Wireshark Protocol Analyzer Software

Many port scanners today are considered IP scanners. IP scanners are superb at running port sweeps against entire subnets of IP addressed network hosts or systems.

In addition to being helpful to network administrators, port scanners can be used to carry out harm. Computer criminals might use port scanners in launching attacks on open ports.

A very famous port scanner tool is a simple command-line tool called netstat. This tool is covered in detail later in this chapter.

### iperf

*iperf* is a software tool for networking that obtains active measurements of the maximum bandwidth available on IP networks. This software tool supports tuning of various parameters related to timing, buffers, and protocols. The protocols you can tune include Transmission Control Protocol (TCP), User Datagram Protocol (UDP), and Stream Control Transmission Protocol (SCTP) with Internet Protocol Version 4 (IPv4) and Version 6 (IPv6).

iperf reports on bandwidth, packet loss, and other important network parameters. A nice feature of iperf is that it can be run in Linux and Windows environments.

### NetFlow Analyzers

NetFlow, which is a software component included in many network devices today, collects statistics about the traffic flows through a device. However, the data that NetFlow collects is not useful unless you can analyze it and see it in a way that makes sense. You can use *NetFlow analyzers* (often also called *collectors*) to examine NetFlow collections and get valuable and insightful representations of the data.

### TFTP Server

For network maintenance, it is helpful to have a *Trivial File Transfer Protocol (TFTP) server* or two on your network. These low-overhead-producing servers allow you to store backups of software images that power network devices, as well as backup and baseline configurations. Many network devices today can install their software or configurations from a TFTP server directly over the network. Be sure to remember security when working with these servers. This is critical because the TFTP server itself does not provide security mechanisms like secure authentication or encryption.

### Terminal Emulator

A software tool you are likely to find yourself using all the time is a *terminal emulator*. This program allows you to access a command-line interface (CLI) to monitor and configure devices, which is often a very efficient way to carry out these networking tasks.

### IP Scanner

Many tools can provide IP address information for systems in a network, but *IP scanners* can do this precise function better than any alternative. IP scanners can acquire information more accurately and faster than the alternatives. Many IP scanners can also provide additional information from nodes, such as operating system, software application, and physical resource usage information.

### LLDP/CDP

***Link Layer Discovery Protocol (LLDP)*** is a network protocol used by devices to identify and share information about themselves and their neighbors within a local area network. Think of it as a way for network devices, like switches and routers, to introduce themselves and exchange details, such as their names, capabilities, and

connections. This helps network administrators map out the network topology, making it easier to manage and troubleshoot connections between devices.

LLDP operates at the data link layer (Layer 2) of the OSI model. It works directly with the hardware that connects devices on a network. It sends out small packets of information, called LLDP Data Units (LLDPDUs), at regular intervals. These packets are received by neighboring devices, which store the information in a special database. By using LLDP, administrators can quickly gather crucial information about the network, such as which devices are connected, how they are connected, and their specific capabilities, all without having to physically inspect the network hardware.

While LLDP is an open standard defined by the Institute of Electrical and Electronics Engineers (IEEE), ***Cisco Discovery Protocol (CDP)*** is an invention by Cisco Systems. It essentially strives to do what LLDP does. CDP helps Cisco devices, like switches and routers, automatically discover and share information with each other within a local network. CDP allows Cisco devices to exchange details such as device identity, capabilities, and the status of interfaces. This helps network administrators easily map out the network, troubleshoot issues, and optimize performance. By regularly sending out small packets of information, CDP ensures that devices can keep each other updated about their presence and capabilities, simplifying network management in environments using Cisco equipment.

**NOTE** Cisco is such a giant in the networking industry that you might encounter non-Cisco devices that also speak CDP. A great example is non-Cisco IP phones that speak CDP so they can seamlessly integrate with Cisco collaboration solutions.



## Command-Line Tools

You will often find yourself firing open a terminal emulator to access a command-line interface when you are maintaining or troubleshooting a network. This section ensures that you are very familiar with many of the command-line tools in use today.

### ping

The ***ping*** command is one of the most commonly used command-line commands. You can use it to check IP connectivity between two network devices. Multiple platforms (for example, routers, switches, and hosts) support the **ping** command. We will focus on the Windows version of this command, but keep in mind there are versions for just about every operating system you can think of.

The **ping** command uses Internet Control Message Protocol (ICMP), which is a Layer 4 protocol. If you issue a **ping** command from your PC, your PC sends an ICMP echo message to the specified destination host. If the destination host is reachable, the host responds with an ICMP echo reply message. Other ICMP messages can be returned to your PC, from your PC's default gateway, to indicate that a destination host is unreachable, that an ICMP echo timed out, or that a Time-to-Live (TTL) value (which is decremented by 1 at each router hop) has expired (that is, has been decremented to a value of 0).

The syntax of the **ping** command, along with some of its commonly used options, is as follows:

```
ping [-t] [-n count] [-l size] [-f] [-i TTL] [-S srcaddr] target_name
```

Table 25-1 explains the **ping** command options.

**Table 25-1** Parameters for the Windows **ping** Command

| Parameter | Purpose |
|---|---|
| **-t** | This option repeatedly sends pings (ICMP echo messages) until you stop it by pressing Ctrl-C. |
| **-n** *count* | This option specifies the number of pings to send. |
| **-l** *size* | This option enables you to define the size of the ICMP echo requests in bytes; the default value is 32. |
| **-f** | This option sets the "don't fragment" bit in a packet's header. If the packet tries to cross a router that attempts to fragment the packet, the packet is dropped, and an ICMP error message is returned. |
| **-i** *TTL* | This option sets the TTL value in a packet's header. The TTL value is decremented for each router hop. A packet is discarded when its TTL value reaches 0. |
| **-S** *srcaddr* | If the PC from which you are issuing the **ping** command has more than one IP address, this option allows you to specify the source IP address from which the ICMP echo messages should be sent. |
| *target_name* | This option specifies the name or the IP address of the device to which you are sending ICMP echo messages. |

A Windows **ping** command specifying only the *target_name* parameter sends four ICMP echo messages to the specified target, as shown in Example 25-1. In the output, notice that none of the packets were dropped.

Key Topic

**Example 25-1** Sample Output from the Windows **ping** Command

```
C:\> ping 192.168.1.2
Pinging 192.168.1.2 with 32 bytes of data:
Reply from 192.168.1.2: bytes=32 time=2ms TTL=64
Reply from 192.168.1.2: bytes=32 time=1ms TTL=64
Reply from 192.168.1.2: bytes=32 time=1ms TTL=64
Reply from 192.168.1.2: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.1.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 2ms, Average = 1ms
```

If the specified target address is unreachable, output from the **ping** command indicates that the target cannot be reached, as shown in Example 25-2.

**Example 25-2** Windows **ping** Command Indicating an Unreachable Destination

```
C:\> ping 192.168.1.200

Pinging 192.168.1.200 with 32 bytes of data:
Reply from 192.168.1.50: Destination host unreachable.
Reply from 192.168.1.50: Destination host unreachable.
Reply from 192.168.1.50: Destination host unreachable.
Reply from 192.168.1.50: Destination host unreachable.

Ping statistics for 192.168.1.200:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
```

### ping with IPv6

Depending on the operating system, **ping** can natively work to test connectivity using IPv6 when an IPv6 destination address is part of the **ping** command. On some systems, the command **ping -6** *IPv6-destination-address*, **ping6** *IPv6-destination-address*, or some variant specific to that operating system may be available for testing IPv6 connectivity.

### ipconfig

You can use the ***ipconfig*** command to display IP address configuration parameters on a Windows PC. In addition, if the PC uses Dynamic Host Configuration Protocol (DHCP), you can use the **ipconfig** command to release and renew a DHCP lease, which is often useful when troubleshooting.

The syntax of the **ipconfig** command, along with some of its most commonly used parameters, is as follows:

```
ipconfig [ /all | /renew | /release | /renew6 | /release6]
```

Table 25-2 describes the parameters for the **ipconfig** command.

**Table 25-2** Parameters for the Windows **ipconfig** Command

| Parameter | Purpose |
|---|---|
| **/all** | The **ipconfig** command entered by itself displays summary information about a PC's IP address configuration. This parameter gives more verbose information, including DNS, MAC address, and IPv6 address information. |
| **/release** and **/release6** | These options release a DHCP lease for an IPv4 and IPv6 address, respectively. |
| **/renew** and **/renew6** | These options renew a DHCP lease for an IPv4 and IPv6 address, respectively. |

Example 25-3 shows the **ipconfig** command, without any parameters, being issued on a PC. The PC contains an Ethernet network interface card (NIC) and a wireless NIC. From the output, you can conclude that one of the NICs has the IP address 172.16.202.129, and the other NIC has the IP address 172.16.202.128. Also, you can see that these two NICs share the common default gateway 172.16.202.2.

Key Topic

**Example 25-3** Sample Output from the Windows **ipconfig** Command

```
C:\> ipconfig
Windows IP Configuration
Ethernet adapter Local Area Connection 3:
   Connection-specific DNS Suffix .     : localdomain
   Link-local IPv6 Address . . . . .    : fe80::5101:b420:4354:d496%20
   IPv4 Address. . . . . . . . . . .    : 172.16.202.129
   Subnet Mask . . . . . . . . . . .    : 255.255.255.0
   Default Gateway . . . . . . . . .    : 172.16.202.2
Ethernet adapter Local Area Connection  :
   Connection-specific DNS Suffix .     : localdomain
   Link-local IPv6 Address . . . . .    : fe80::a10f:cff4:15e4:aa6%11
   IPv4 Address. . . . . . . . . . .    : 172.16.202.128
   Subnet Mask . . . . . . . . . . .    : 255.255.255.0
   Default Gateway . . . . . . . . .    : 172.16.202.2
OUTPUT OMITTED...
```

Example 25-4 shows the **ipconfig /all** command being issued on a PC. Notice the additional output from this command beyond what is shown for the **ipconfig** command in Example 25-3. For example, in Example 25-4 you can see the MAC address (labeled as the *physical address*) for each NIC and the DNS server's IP address, 172.16.202.2.

**Example 25-4** Sample Output from the Windows **ipconfig /all** Command

```
C:\> ipconfig /all
Windows IP Configuration
   Host Name . . . . . . . . . . . . : WIN-OD1IG7JF47P
   Primary Dns Suffix . . . . . . . : 
   Node Type . . . . . . . . . . . . : Hybrid
   IP Routing Enabled. . . . . . . . : No
   WINS Proxy Enabled. . . . . . . . : No
   DNS Suffix Search List. . . . . . : localdomain
Ethernet adapter Local Area Connection 3:
   Connection-specific DNS Suffix . . : localdomain
   Description . . . . . . . . . . . : Intel(R) PRO/1000 MT Network
                                       Connection #2
   Physical Address. . . . . . . . . : 00-0C-29-3A-21-67
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
   Link-local IPv6 Address . . . . . : fe80::5101:b420:4354:d496%20
                                       (Preferred)
   IPv4 Address. . . . . . . . . . . : 172.16.202.129(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Lease Obtained. . . . . . . . . . : Saturday, May 29, 2026 6:28:08
                                       PM
   Lease Expires . . . . . . . . . . : Saturday, May 29, 2026 9:28:08
                                       PM
   Default Gateway . . . . . . . . . : 172.16.202.2
   DHCP Server . . . . . . . . . . . : 172.16.202.254
   DHCPv6 IAID . . . . . . . . . . . : 419433513
   DHCPv6 Client DUID. . . . . . . . : 00-01-00-01-14-A6-11-77-00-0C-
                                       29-3A-21-5D
   DNS Servers . . . . . . . . . . . : 172.16.202.2
   Primary WINS Server . . . . . . . : 172.16.202.2
   NetBIOS over Tcpip. . . . . . . . : Enabled
Ethernet adapter Local Area Connection:
   Connection-specific DNS Suffix . . : localdomain
```

```
   Description . . . . . . . . . . . : Intel(R) PRO/1000 MT Network
                                       Connection
   Physical Address. . . . . . . . . : 00-0C-29-3A-21-5D
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
   Link-local IPv6 Address . . . . . : fe80::a10f:cff4:15e4:aa6%11
                                       (Preferred)
   IPv4 Address. . . . . . . . . . . : 172.16.202.128(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Lease Obtained. . . . . . . . . . : Saturday, May 29, 2026
                                       6:27:56 PM
   Lease Expires . . . . . . . . . . : Saturday, May 29, 2026 9:28:08
                                       PM
   Default Gateway . . . . . . . . . : 172.16.202.2
   DHCP Server . . . . . . . . . . . : 172.16.202.254
   DHCPv6 IAID . . . . . . . . . . . : 234884137
   DHCPv6 Client DUID. . . . . . . . : 00-01-00-01-14-A6-11-77-00-0C-
                                       29-3A-21-5D
   DNS Servers . . . . . . . . . . . : 172.16.202.2
   Primary WINS Server . . . . . . . : 172.16.202.2
   NetBIOS over Tcpip. . . . . . . . : Enabled
OUTPUT OMITTED...
```

If you are troubleshooting a PC and suspect that IP addressing might be an issue, you can release the PC's current DHCP lease by issuing the **ipconfig /release** command, as shown in Example 25-5. Then you can renew the DHCP lease with the **ipconfig /renew** command, as shown in Example 25-6.

**Example 25-5** Sample Output from the Windows **ipconfig /release** Command

```
C:\> ipconfig /release
Windows IP Configuration
Ethernet adapter Local Area Connection 3:
   Connection-specific DNS Suffix . . . :
   Link-local IPv6 Address . . . . . : fe80::5101:b420:4354:d496%20
   Default Gateway . . . . . . . . . :
Ethernet adapter Local Area Connection  :
   Connection-specific DNS Suffix . . . :
   Link-local IPv6 Address . . . . . :  fe80::a10f:cff4:15e4:aa6%11
   Default Gateway . . . . . . . . . :
OUTPUT OMITTED...
```

**Example 25-6** Sample Output from the Windows **ipconfig /renew** Command

```
C:\> ipconfig /renew
Windows IP Configuration
Ethernet adapter Local Area Connection 3:
   Connection-specific DNS Suffix . . . :  localdomain
   Link-local IPv6 Address . . . . . : fe80::5101:b420:4354:d496%20
   IPv4 Address. . . . . . . . . . . :  172.16.202.129
   Subnet Mask . . . . . . . . . . . :  255.255.255.0
   Default Gateway . . . . . . . . . :  172.16.202.2
Ethernet adapter Local Area Connection:
   Connection-specific DNS Suffix . . . :  localdomain
   Link-local IPv6 Address . . . . . :  fe80::a10f:cff4:15e4:aa6%11
   IPv4 Address. . . . . . . . . . . :  172.16.202.128
   Subnet Mask . . . . . . . . . . . :  255.255.255.0
   Default Gateway . . . . . . . . . :  172.16.202.2
OUTPUT OMITTED...
```

### ifconfig

The rough equivalent in the UNIX/Linux world for the **ipconfig** utility is ***ifconfig***. Example 25-7 demonstrates the use of this command to learn IP address information configured on an interface.

**Example 25-7** Sample Output from the Linux **ifconfig** Command

```
anthony@DESKTOP-91165JO:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.86.29  netmask 255.255.255.0  broadcast
192.168.86.255
        inet6 fe80::6044:94be:583f:4f82  prefixlen 64  scopeid
0xfd<compat,link,site,host>
        ether 84:8f:69:f5:5f:3d  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 1500
        inet 127.0.0.1  netmask 255.0.0.0
```

```
        inet6 ::1  prefixlen 128  scopeid 0xfe<compat,link,site,host>
        loop  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
anthony@DESKTOP-91165JO:~$
```

**NOTE** This might sound crazy, but I captured Example 25-7 on my Windows 11 desktop PC. It is simple now to run Linux inside Windows.

### ip

Many Linux systems are moving to the ***ip*** command instead of even preloading the **ifconfig** command. The powerful **ip** command functions with many additional keywords, including the **address** keyword, which displays the same information as **ifconfig**.

### nslookup

Although the ***nslookup*** command offers various command options, this section focuses on the most common use for the command: resolving a fully qualified domain name (FQDN) to an IP address. This can, for example, help you determine whether a DNS record is correct and verify that your DNS server is operating.

The **nslookup** command can be issued along with an FQDN, or it can be used in an interactive mode, where you are prompted to enter command parameters. The syntax of this command can be summarized as follows:

**nslookup** [*fqdn*]

In noninteractive mode, you issue the **nslookup** command followed by an FQDN to display the IP address corresponding to the FQDN. To illustrate, consider Example 25-8, where the **nslookup** command is issued to resolve the IP address of the website ajsnetworking.com, which appears to be 172.31.194.74.

**NOTE** A private IP address is used here for illustrative purposes. In a real-world example, a public IP address would be displayed.

Key Topic

**Example 25-8** Sample Output from the Windows **nslookup** Noninteractive Command

```
C:\> nslookup ajsnetworking.com
Server: UnKnown
Address: 192.168.1.1

Non-authoritative answer:
Name: ajsnetworking.com
Address: 172.31.194.74
```

In interactive mode, you enter the **nslookup** command and then, from the > prompt, enter command parameters. Example 25-9 shows ajsnetworking.com entered at the prompt to see the IP address corresponding to that FQDN. Also, notice that entering a question mark (**?**) displays a help screen that shows command options. By entering **quit**, you exit interactive mode.

**Example 25-9** Sample Output from the Windows **nslookup** Interactive Command

```
C:\> nslookup
Default Server: UnKnown
Address: 192.168.1.1

> ajsnetworking.com
Server: UnKnown
Address: 192.168.1.1

Non-authoritative answer:
Name: ajsnetworking.com
Address: 172.31.194.74

> ?
Commands:   (identifiers are shown in uppercase, [] means optional)
NAME        - print info about the host/domain NAME using default
server
NAME1 NAME2            - as above, but use NAME2 as server
help or ?         - print info on common commands
set OPTION            - set an option
   all                              - print options, current server
and host
   [no]debug                - print debugging information
```

```
    [no]d2                                  - print exhaustive debugging
information
    [no]defname                    - append domain name to each query
    [no]recurse               - ask for recursive answer to query
    [no]search                                - use domain search list
    [no]vc                              - always use a virtual circuit
    domain=NAME               - set default domain name to NAME
    srchlist=N1[/N2/.../N6]    - set domain to N1 and search list to
N1,N2, etc.
    root=NAME                                - set root server to NAME
OUTPUT OMITTED...
> quit
C:\>
```

### dig

As you have seen here, the Windows **nslookup** command is used to resolve a given FQDN to its IP address. UNIX has a similar **nslookup** command, which you can also use for FQDN to IP address resolution.

You can also use the ***dig*** command to resolve FQDNs to IP addresses. Unlike the **nslookup** command, however, the **dig** command is entirely a command-line command. (That is, **dig** lacks the interactive mode of the **nslookup** command.)

Example 25-10 compares the output of the **nslookup** and **dig** commands. Notice that the **dig** command offers more information than the **nslookup** command. For example, the *A* in the QUESTION SECTION part of the output of the **dig** command identifies the DNS record type (an A record, which is an alias record). If you peruse the output, you can find a few other pieces of information present in the **dig** command output that are not available in the **nslookup** command output; however, the **dig** command is rarely used to glean these more subtle pieces of information. Rather, many UNIX administrators use the **dig** command simply as an alternative way of resolving FQDNs to IP addresses. Notice in Example 25-10 that both the **nslookup** and **dig** commands indicate that the IP address corresponding to the FQDN www.pearsonitcertification.com is 159.182.74.51.

Key Topic

**Example 25-10** Comparing Output from the Windows **nslookup** and UNIX **dig** Commands

```
C:\> nslookup www.pearsonitcertification.com
Server: 192.168.1.1
Address: 192.168.1.1#53
```

```
Non-authoritative answer:

Name: www.pearsonitcertification.com
Address: 159.182.74.51

HOST# dig www.pearsonitcertification.com

; <<>> DiG 9.6.0-APPLE-P2 <<>> www.pearsonitcertification.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 10821
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;www.pearsonitcertification.com. IN A

;; ANSWER SECTION:
www.pearsonitcertification.com. 10791 IN A 159.182.74.51

;; Query time: 5 msec
;; SERVER: 192.168.1.1#53(192.168.1.1)
;; WHEN: Mon May 30 13:36:11 2026
;; MSG SIZE rcvd: 64
```

### traceroute

You can use the UNIX command ***traceroute*** to determine which router hop along the path from a source device to a destination device is having issues. Also, based on the round-trip response time information reported for each hop, you can better determine which network segment might be causing excessive delay due to congestion. Example 25-11 provides sample output from the **traceroute** command, which is identifying the 13 router hops a UNIX host must transit to reach pearsonitcertification.com.

Key Topic

**Example 25-11** Sample Output from the UNIX **traceroute** Command

```
HOST# traceroute www.pearsonitcertification.com
traceroute to pearsonitcertification.com (64.28.85.25), 64 hops max,
52 byte packets
 1  192.168.1.1 (192.168.1.1) 3.480 ms 2.548 ms 2.404 ms
```

```
 2  cpe-76-177-16-1.natcky.res.rr.com (76.177.16.1) 22.150 ms
11.300 ms 9.719 ms
 3  gig2-0-0.rcmdky-mx41.natcky.rr.com (65.28.199.205) 9.242 ms
19.940 ms 11.735 ms
 4  tge0-2-0.chcgileq-rtr1.kc.rr.com (65.28.199.97) 38.459 ms
38.821 ms 36.157 ms
 5  ae-4-0.cr0.chi10.tbone.rr.com (66.109.6.100) 41.903 ms 37.388 ms
31.966 ms
 6  ae-0-0.pr0.chi10.tbone.rr.com (66.109.6.153) 75.757 ms 46.287 ms
35.031 ms
 7  if-4-0-0.core1.ct8-chicago.as6453.net (66.110.14.21) 48.020 ms
37.248 ms 45.446 ms
 8  if-1-0-0-1878.core2.ct8-chicago.as6453.net (66.110.27.78)
108.466 ms 55.465 ms 87.590 ms
 9  63.243.186.25 (63.243.186.25) 64.045 ms 63.582 ms 69.200 ms
10  cr2-pos-0-8-0-3.nyr.savvis.net (208.173.129.29) 64.933 ms
65.113 ms 61.759 ms
11  hr1-tengig-13-0-0.waltham2bo2.savvis.net (204.70.198.182)
71.964 ms 65.430 ms 74.397 ms
12  das3-v3038.bo2.savvis.net (209.202.187.182) 65.777 ms 64.483 ms
82.383 ms
13  blhosting.bridgelinesw.com (64.14.81.46) 63.448 ms !X * 68.879 ms
!X
```

The Windows ***tracert*** command can be used for the same purpose as the UNIX **traceroute** command.

### traceroute for IPv6

You can verify the IPv6 path through a network by using **traceroute** for IPv6. Depending on the vendor and platform, this may be done by using **traceroute** *destination-IPv6-address*, **traceroute6** *destination-IPv6-address*, **traceroute -6** *destination-IPv6-address*, or some variant specific to the vendor and product being used.

### arp

You can use the ***arp*** command to view the MAC address to IP address name resolution that has succeeded and been entered in the ARP cache. In addition, you can use the **arp** command to statically add a MAC address to IP address mapping to a PC's Address Resolution Protocol (ARP) MAC address lookup table.

The syntax of the **arp** command is as follows:

**arp -s** *inet_addr eth_addr* [*if_addr*]
**arp -d** *inet_addr* [*if_addr*]
**arp -a** [*inet_addr*] [**-N** *if_addr*] [**-v**]

Table 25-3 describes the **arp** command's *switches* (for example, **-s**, **-d**, and **-a**) and *arguments* (for example, *inet_addr* and *if_addr*).

**Table 25-3** Parameters for the Windows **arp** Command

| Parameter | Purpose |
|---|---|
| **-a** or **-g** | These options display current entries in a PC's ARP table. |
| **-v** | This option, which stands for *verbose*, includes any invalid and loopback interface entries in an ARP table. |
| *inet_addr* | This option is a specific IP address. |
| **-N** *if_addr* | This option shows ARP entries learned for a specified network. |
| **-d** | This option, in combination with the *inet_addr* parameter, can delete an ARP entry for a host. With the wildcard character *, it can delete all host entries. |
| **-s** | This option, used in conjunction with the *inet_addr* and *eth_addr* parameters, statically adds a host entry to the ARP table. |
| *eth_addr* | This parameter is a 48-bit MAC address. |
| *if_addr* | If a host has multiple interfaces, an ARP entry might be associated with a specific interface. This option can be used to statically add or delete an ARP entry in a specified interface. |

Example 25-12 shows the **arp -a** command being issued on a PC. The output shows what MAC addresses have been learned for the listed IP addresses. The dynamically learned addresses have *dynamic* listed in the *Type* column, and statically configured addresses (which are addresses configured by a user or the OS) are listed with *static* in the *Type* column. From the output, you can determine, for example, that the network device with the IP address 172.16.202.1 has the MAC address 00-50-56-c0-00-08, which could alternatively be written as 0050.56c0.0008. Also, you can determine from the output that this information was dynamically learned, as opposed to being statically configured.

Key Topic

**Example 25-12** Sample Output from the Windows **arp -a** Command

```
C:\> arp -a
Interface: 172.16.202.128 --- 0xb
   Internet Address      Physical Address      Type
   172.16.202.1          00-50-56-c0-00-08     dynamic
   172.16.202.2          00-50-56-fd-65-2c     dynamic
```

```
  172.16.202.254       00-50-56-e8-84-fc       dynamic
  172.16.202.255       ff-ff-ff-ff-ff-ff       static
  224.0.0.22                    01-00-5e-00-00-16       static
  224.0.0.252                   01-00-5e-00-00-fc       static
  255.255.255.255      ff-ff-ff-ff-ff-ff       static

Interface: 172.16.202.129 --- 0x14
  Internet Address      Physical Address        Type
  172.16.202.1                  00-50-56-c0-00-08       dynamic
  172.16.202.2                  00-50-56-fd-65-2c       dynamic
  172.16.202.254       00-50-56-e8-84-fc       dynamic
  172.16.202.255       ff-ff-ff-ff-ff-ff       static
  224.0.0.22                    01-00-5e-00-00-16       static
  224.0.0.252                   01-00-5e-00-00-fc       static
  224.0.1.60                    01-00-5e-00-01-3c       static
  255.255.255.255      ff-ff-ff-ff-ff-ff       static
```

From a troubleshooting perspective, keep in mind that static ARP entries tend to be more problematic than dynamic entries. For example, a static entry might be added to a laptop computer, and the computer might later connect to a different network. If a PC then attempts to reach the IP address specified in the static ARP entry, the Layer 2 frame would have the incorrect destination MAC address (which should then be the MAC address of the PC's default gateway) in its header.

### netstat

You can use the ***netstat*** command to display various information about IP-based connections on a PC. For example, you can view information about current sessions, including source and destination IP addresses and port numbers. You can also display protocol statistics, which might be useful for troubleshooting purposes. For example, you might issue the **netstat** command and see that your PC has sessions open to an unknown host on the Internet. These sessions might prompt further investigation to determine why the sessions are open and if they might be resulting in performance issues on the PC or possibly posing a security risk.

The following is the syntax for the **netstat** command and some of its commonly used options:

**netstat** [**-a**] [**-b**] [**-e**] [**-f**] [**-p** *proto*] [**-r**] [**-s**]

Table 25-4 explains these command options.

**Table 25-4** Parameters for the Windows **netstat** Command

| Parameter | Purpose |
|---|---|
| **-a** | This option displays all of a PC's active IP-based sessions, along with the TCP and UDP ports of each session. |
| **-b** | This option shows the name of the program that opened a session. |
| **-e** | This option shows statistical information for an interface's IP-based traffic, such as the number of bytes sent and received. |
| **-f** | This option displays FQDNs of destination addresses appearing in a listing of active sessions. |
| **-p** *proto* | This option displays connections for a specific protocol, which might be **icmp**, **icmpv6**, **ip**, **ipv6**, **tcp**, **tcpv6**, **udp**, or **udpv6**. |
| **-r** | This option displays a PC's IP routing table. (Note that **netstat** with this option generates the same output as the **route print** command.) |
| **-s** | This option displays statistical information for the following protocols: ICMPv4, ICMPv6, IPv4, IPv6, TCPv4, TCPv6, UDPv4, and UDPv6. |

The **netstat** command issued without any options lists source and destination IP addresses and port numbers for all IP-based sessions. Example 25-13 shows sample output from this command.

Key Topic

**Example 25-13** Sample Output from the Windows **netstat** Command

```
C:\> netstat
OUTPUT OMITTED...
  TCP   127.0.0.1:27015      LIVE-DELIVERY:1309            ESTABLISHED
  TCP   192.168.1.50:1045    172.16.224.200:https          CLOSE_WAIT
  TCP   192.168.1.50:1058    THE-WALLACES-TI:microsoft-ds  ESTABLISHED
  TCP   192.168.1.50:1079    tcpep:https                   ESTABLISHED
  TCP   192.168.1.50:1081    174:http                      ESTABLISHED
  TCP   192.168.1.50:1089    by2msg4020609:msnp            ESTABLISHED
  TCP   192.168.1.50:1111    HPB81308:netbios-ssn          ESTABLISHED
  TCP   192.168.1.50:1115    10.65.228.81:https            ESTABLISHED
  TCP   192.168.1.50:1116    10.65.228.81:https            ESTABLISHED
  TCP   192.168.1.50:1117    10.65.228.81:https            ESTABLISHED
  TCP   192.168.1.50:1118    10.65.228.81:https            ESTABLISHED
  TCP   192.168.1.50:1126    10.65.228.81:https            ESTABLISHED
  TCP   192.168.1.50:1417    vip1:http                     CLOSE_WAIT
  TCP   192.168.1.50:1508    208:https                     CLOSE_WAIT
  TCP   192.168.1.50:1510    208:https                     CLOSE_WAIT
  TCP   [::1]:2869           LIVE-DELIVERY:1514            TIME_WAIT
  TCP   [::1]:286            LIVE-DELIVERY:1515            ESTABLISHED
OUTPUT OMITTED...
```

You might notice an open connection using a specific port and be unsure what application opened that connection. As you can see in Example 25-14, the **netstat -b** command shows which application opened a specific connection. In this example, Dropbox.exe, iTunes.exe, firefox.exe, and OUTLOOK.exe are applications that have currently open connections.

Key Topic

**Example 25-14** Sample Output from the Windows **netstat -b** Command

```
C:\> netstat -b
Active Connections
OUTPUT OMITTED...
 Proto           Local Address       Foreign Address            State
 TCP             127.0.0.1:1068      LIVE-DELIVERY:19872    ESTABLISHED
[Dropbox.exe]
 TCP             127.0.0.1:1309      LIVE-DELIVERY:27015    ESTABLISHED
[iTunes.exe]
 TCP             127.0.0.1:1960      LIVE-DELIVERY:1961     ESTABLISHED
[firefox.exe]
 TCP             192.168.1.50:1115   10.1.228.81:https      ESTABLISHED
[OUTLOOK.EXE]
 TCP             192.168.1.50:1116   10.1.228.81:https      ESTABLISHED
[OUTLOOK.EXE]
OUTPUT OMITTED...
```

### hostname

The **hostname** command is available on both UNIX/Linux and Windows systems. On a Linux system, you use this command to change the computer name or the domain name of the system. When you run the command on a Windows system, the command simply displays the computer name.

### route

The **route** command can display a PC's current IP routing table. In addition, you can use the **route** command to add or delete entries in the routing table. The syntax of the **route** command, with a collection of commonly used options, is as follows:

```
C:\> route [-f] [-p] command [destination] [mask netmask] [gateway]
[metric metric] [if interface]
```

Table 25-5 explains these command options.

**Table 25-5** Parameters for the Windows **route** Command

| Parameter | Purpose |
|---|---|
| **-f** | This option clears gateway entries from the routing table. If this option is used with another option, the clearing of gateways from the routing table occurs before any other specified action. |
| **-p** | This option can be used with the **add** command to make a statically configured route persistent, which means the route will remain in a PC's routing table even after a reboot. |
| *command* | Supported commands include **print**, **add**, **delete**, and **change**. The **print** command lists entries in a PC's routing table. The **add** command adds a route entry. The **delete** command removes a route from the routing table, and the **change** command can modify an existing route. |
| *destination* | This option specifies the destination host or subnet to add to a PC's routing table. |
| **mask** *netmask* | This option, used in conjunction with the *destination* option, specifies the subnet mask of the destination. If the destination is the IP address of a host, the *netmask* parameter is 255.255.255.255. |
| *gateway* | This option specifies the IP address of the next-hop router used to reach the specified destination. |
| **metric** *metric* | This option specifies the cost to reach a specified destination. If a routing table contains more than one route to reach the destination, the route with the lowest cost is selected. |
| **if** *interface* | This option forwards traffic to a specified destination out a specific interface. |

Example 25-15 illustrates the use of the **route print** command, which displays the contents of a PC's routing table. Notice that the output identifies a listing of the PC's interfaces, along with IPv4 routes and IPv6 routes. From the output, you can see that the 10.0.0.0 255.0.0.0 network is reachable via two gateways (192.168.1.77 and 192.168.1.11). Also, notice that there is a persistent route (a route entry that survives a reboot) to act as a default gateway for the PC, which is 192.168.1.1.

**Example 25-15** Sample Output from the Windows **route print** Command

```
C:\> route print
===========================================================================
Interface List
 11...00 24 81 ee 4c 0e ......Intel(R) 82566DM-2 Gigabit Network
Connection
  1...........................Software Loopback Interface 1
```

```
 12...00 00 00 00 00 00 00 e0 Microsoft ISATAP Adapter
 13...00 00 00 00 00 00 00 e0 Teredo Tunneling Pseudo-Interface
===========================================================================

IPv4 Route Table
===============================+=======================================
Active Routes:
Network Destination  Netmask          Gateway       Interface       Metric
    0.0.0.0          0.0.0.0          192.168.1.1   192.168.1.50    276
    10.0.0.0         255.0.0.0        192.168.1.77  192.168.1.50    21
    10.0.0.0         255.0.0.0        192.168.1.11  192.168.1.50    21
    127.0.0.0        255.0.0.0        On-link       127.0.0.1       306
    127.0.0.1        255.255.255.255  On-link       127.0.0.1       306
    127.255.255.255  255.255.255.255  On-link       127.0.0.1       306
    172.16.0.0       255.255.0.0      192.168.1.1   192.168.1.50    21
    192.168.0.0      255.255.255.0    192.168.1.11  192.168.1.50    21
    192.168.1.0      255.255.255.0    On-link       192.168.1.50    276
    192.168.1.50     255.255.255.255  On-link       192.168.1.50    276
    192.168.1.255    255.255.255.255  On-link       192.168.1.50    276
    224.0.0.0        240.0.0.0        On-link       127.0.0.1       306
    224.0.0.0        240.0.0.0        On-link       192.168.1.50    276
    255.255.255.255  255.255.255.255  On-link       127.0.0.1       306
    255.255.255.255  255.255.255.255  On-link       192.168.1.50    276
===========================================================================
Persistent Routes:
    Network Address  Netmask      Gateway Address   Metric
    0.0.0.0          0.0.0.0      192.168.1.1       Default
===========================================================================

IPv6 Route Table
===========================================================================
Active Routes:
 If   Metric     Network Destination           Gateway

 13   58 :      :/0                           On-link
 1    306 :     :1/128                        On-link
 13   58        2001::/32 On-link
 13   306       2001:0:4137:9e76:10e2:614f:b34e:ea84/128
                                              On-link
```

```
 11    276      fe80::/64                     On-link
 13    306      fe80::/64                     On-link
 13    306      fe80::10e2:614f:b34e:ea84/128
                                              On-link
 11    276      fe80::f46d:4a34:a9c4:51a0/128
                                              On-link
 1     306      ff00::/8                      On-link
 13    306      ff00::/8                      On-link
 11    276      ff00::/8                      On-link
===========================================================================
Persistent Routes:
  None
```

Say that you want to remove one of the route entries for the 10.0.0.0 255.0.0.0 network. Example 25-16 shows how one of the two entries (specifically, the entry pointing to 192.168.1.11) can be removed from the routing table. Notice from the output that after the **route delete 10.0.0.0 mask 255.0.0.0 192.168.1.11** command is issued, the route no longer appears in the routing table.

**Example 25-16** Sample Output from the Windows **route delete** Command

```
C:\> route delete 10.0.0.0 mask 255.0.0.0 192.168.1.11
 OK!
C:\> route print
OUTPUT OMITTED...
IPv4 Route Table
===========================================================++++=================
Active Routes:
Network Destination  Netmask          Gateway       Interface      Metric
0.0.0.0              0.0.0.0          192.168.1.1   192.168.1.50   276
10.0.0.0             255.0.0.0        192.168.1.77  192.168.1.50   21
127.0.0.0            255.0.0.0        On-link       127.0.0.1      306
127.0.0.1            255.255.255.255  On-link       127.0.0.1      306
127.255.255.255      255.255.255.255  On-link       127.0.0.1      306
172.16.0.0           255.255.0.0      192.168.1.11  192.168.1.50   21
192.168.0.0          255.255.255.0    192.168.1.11  192.168.1.50   21
192.168.1.0          255.255.255.0    On-link       192.168.1.50   276
```

```
192.168.1.50        255.255.255.255  On-link      192.168.1.50   276
192.168.1.255       255.255.255.255  On-link      192.168.1.50   276
224.0.0.0           240.0.0.0        On-link      127.0.0.1      306
224.0.0.0           240.0.0.0        On-link      192.168.1.50   276
255.255.255.255     255.255.255.255  On-link      127.0.0.1      306
255.255.255.255     255.255.255.255  On-link      192.168.1.50   276
===========================================================================
OUTPUT OMITTED...
```

You can add a route by using the **route add** command. Example 25-17 shows and confirms the addition of a route pointing to the 10.2.1.0 255.255.255.0 network, with the next-hop route (gateway) 192.168.1.1.

**Example 25-17** Sample Output from the Windows **route add** Command

```
C:\> route add 10.2.1.0 mask 255.255.255.0 192.168.1.1
 OK!
C:\> route print
OUTPUT OMITTED...
IPv4 Route Table
===========================================================================
Active Routes:
Network          Netmask          Gateway       Interface  Metric Destination
0.0.0.0          0.0.0.0          192.168.1.1   192.168.1.50       276
10.0.0.0         255.0.0.0        192.168.1.77 192.168.1.50       21
10.2.1.0         255.255.255.0    192.168.1.1   192.168.1.50       21
127.0.0.0        255.0.0.0        On-link       127.0.0.1          306
127.0.0.1        255.255.255.255  On-link       127.0.0.1          306
127.255.255.255 255.255.255.255  On-link       127.0.0.1          306
172.16.0.0       255.255.0.0      192.168.1.11 192.168.1.50       21
192.168.0.0      255.255.255.0    192.168.1.11 192.168.1.50       21
192.168.1.0      255.255.255.0    On-link       192.168.1.50       276
192.168.1.50     255.255.255.255   On-link      192.168.1.50       276
192.168.1.255    255.255.255.255   On-link      192.168.1.50       276
224.0.0.0        240.0.0.0        On-link       127.0.0.1          306
224.0.0.0        240.0.0.0        On-link       192.168.1.50       276
255.255.255.255 255.255.255.255  On-link       127.0.0.1          306
255.255.255.255 255.255.255.255  On-link       192.168.1.50       276
===========================================================================
OUTPUT OMITTED...
```

### telnet

For many years, **telnet** was the command-line tool of choice for making remote access connections to systems for management. Today, this command is frowned upon because the Telnet protocol offers no security for the information transmitted. Telnet is now found only in lab and practice environments. Secure Shell (SSH) is the technology of choice today for remote access to systems because it is secure.

### tcpdump

You can use the ***tcpdump*** command to print out the headers of packets on a network interface that match a Boolean expression. You can also run the command with the **-w** flag to save the packet data to a file for later analysis and/or with the **-r** flag to read from a saved packet file rather than to read packets from a network interface. The syntax and options for the **tcpdump** command are as follows:

```
tcpdump [ -adeflnNOpqRStuvxX ] [ -c count ] [ -C file_size ] [ -F
file ]
   [ -i interface ] [ -m module ] [ -r file ]
   [ -s snaplen ] [ -T type ] [ -U user ] [ -w file ]
   [ -E algo:secret ] [ expression ]
```

### nmap

The ***nmap*** command (short for Network Mapper) is an open source and very versatile tool for network administrators. While **nmap** was first introduced with Linux systems, this command is now available on Linux, macOS, and Windows systems. You use **nmap** to explore networks, perform security scans, create network audits, and find open ports on remote machines. The tool can scan for live hosts, operating systems, packet filters, and open ports. Here is an example of the syntax for this command:

```
nmap [Scan Type(s)] [Options] {target specification}
```

## Key Topic Basic Networking Device Commands

Although the CompTIA Network+ exam is vendor neutral, you need to be aware of several Cisco network device commands that are incredibly popular and used on equipment from a number of vendors:

- **show interface:** The ***show interface*** command enables you to examine the statistics and the status of the interfaces on a network system.

- **show config:** The ***show config*** command (or some variation of it) is used to examine the configuration of a network device. For example, on a Cisco router, the **show running-configuration** command permits you to see the current configuration of the device, which is stored in the RAM of the device. To view the saved configuration that is loaded when the system is rebooted, you can use the **show startup-configuration** command.
- **show route:** The ***show route*** command (or some variation of it) is used to view the routing table configuration of the network device. On a Cisco router, you can use **show ip route** to view the IPv4 routing table.
- **show mac-address-table:** The ***show mac-address-table*** command allows you to see the MAC address table that currently exists on a Layer 2 or multilayer switch. The MAC address table maintains the mappings of MAC addresses to interfaces on the device. This is the critical "brains" of the switch functionality, allowing it to effectively direct traffic to interfaces using the table mappings.
- **show arp:** Earlier in this chapter, you learned about the **arp** command. On Cisco, and many other networking devices, the ***show arp*** command is the rough equivalent of the **arp** command. The **show arp** command allows you to view the current ARP resolution protocol table of MAC to IP address mappings.
- **show vlan:** On switches, the ***show vlan*** command enables you to view the current VLAN database.
- **show power:** The ***show power*** command on a network device displays information about the device's power status and usage. It helps administrators check how much power the device is consuming and whether there are any power issues or shortages. This is especially important on network devices that are providing Power over Ethernet (PoE) via the device interfaces.

## Hardware Tools

This chapter would not be complete without a discussion of some of the more popular hardware tools that can assist when troubleshooting the network.

### Wi-Fi Analyzer

A ***Wi-Fi analyzer*** is software that runs on a general-purpose computer or on a specialized device that can perform wireless analysis of Wi-Fi signals. This type of tool would be used as part of a wireless site survey after Wi-Fi has been implemented to

create a heat map of the wireless airspace. Such analysis can be critical to ensure network administrators have a thorough knowledge of the wireless infrastructure. Such tools can directly assist with the proper placement of wireless access points and the type and strength of antennas that are needed.

### Tone Generator

If you are working on a punchdown block and attempting to identify which pair of wires connect back to an end user's location (for example, someone's office), you can use a ***toner*** probe. A toner probe allows you to place a tone generator at one end of a connection (for example, someone's office) and use a probe on a punchdown block to audibly detect to which pair of wires the tone generator is connected. A toner probe, therefore, comes in two pieces: the tone generator and the probe. Another common name for a toner probe is a fox and hound, where the tone generator is the fox, and the probe (which searches for the tone) is the hound. Some network devices have built-in troubleshooting tools; for example, a voice-enabled Cisco router can produce test tones.

### Cable Tester

A ***cable tester*** can test the conductors in an Ethernet cable. By connecting the two parts of a cable tester to the ends of a cable under test, you can check the wires in the cable for continuity (that is, check to make sure there are no opens, or breaks, in a conductor). In addition, you can verify an RJ45 connector's pinouts (that is, ensure that wires are connected to the appropriate pins on an RJ45 connector).

### Tap

A traffic access point (also called a test access point or ***tap***) is a hardware device inserted at a specific point in a network where data can be accessed for testing or troubleshooting purposes. Network taps are mainly used to monitor the network traffic between two points in a network infrastructure.

### Visual Fault Locator

A ***visual fault locator*** is a small, handheld device used to find breaks or problems in fiber-optic cables. It works by sending a bright red laser light through the cable. If there is a break or a fault, the light will escape and become visible at the damaged spot. This makes it easy to see where the problem is so it can be fixed quickly.

## Real-World Case Study

Acme, Inc. has invested in several new software packages to help support the network. These include enterprise-level versions of protocol analyzers and port scanners.

Acme, Inc. is also providing training to the junior technicians that includes command-line tool training. This training includes common network device command-line work, including the monitoring commands used in the wide array of Cisco and Juniper network devices in use.

Acme is also updating and expanding on the number of physical troubleshooting kits for the network engineers onsite. These kits include cable testers and Wi-Fi analyzers.

## Summary

The main topics covered in this chapter are the following:

- This chapter covered many different software tools that can assist greatly in the support of the network. This coverage included tools such as Wi-Fi analyzers and port scanners.
- This chapter described command-line tools that you can use to help troubleshoot and maintain modern networks. These included the commonly used tools like **ipconfig** and **traceroute**.
- This chapter covered some basic commands found on some network devices.
- Finally, this chapter covered some of the commonly used hardware tools used to troubleshoot networks.

# Exam Preparation Tasks

## Review All the Key Topics

Review the most important topics from this chapter, noted with the Key Topic icon in the outer margin of the page. Table 25-6 lists these key topics and the page number where each is found.

Key Topic

**Table 25-6** Key Topics for Chapter 25

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 25-1 | Wireshark Protocol Analyzer Software | 545 |
| Section | Command-Line Tools | 547 |
| Example 25-1 | Sample Output from the Windows **ping** Command | 549 |
| Example 25-3 | Sample Output from the Windows **ipconfig** Command | 550 |
| Example 25-8 | Sample Output from the Windows **nslookup** Noninteractive Command | 555 |
| Example 25-10 | Comparing Output from the Windows **nslookup** and UNIX **dig** Commands | 556 |
| Example 25-11 | Sample Output from the UNIX **traceroute** Command | 557 |
| Example 25-12 | Sample Output from the Windows **arp -a** Command | 559 |
| Example 25-13 | Sample Output from the Windows **netstat** Command | 561 |
| Example 25-14 | Sample Output from the Windows **netstat -b** Command | 562 |
| Section | Basic Network Platform Commands | 567 |

## Complete Tables and Lists from Memory

Print a copy of Appendix B, "Memory Tables," or at least the section for this chapter, and complete as many of the tables as possible from memory. Appendix C, "Memory Tables Answer Key," includes the completed tables and lists so you can check your work.

## Define Key Terms

Define the following key terms from this chapter and check your answers in the Glossary:

**arp**, cable tester, Cisco Discovery Protocol (CDP), **dig**, **ifconfig**, **ip**, **ipconfig**, Link Layer Discovery Protocol (LLDP), **netstat**, **nmap**, **nslookup**, **ping**, protocol analyzer, **show arp**, **show config**, **show interface**, **show mac-address-table**, **show power**, **show route**, **show vlan**, speed tester, tap (test access point), **tcpdump**, toner, **traceroute**, **tracert**, visual fault locator, Wi-Fi analyzer

## Additional Resources

**Networking Tools – Hardware:** https://www.youtube.com/watch?v=HAceoquch1c

**Troubleshooting from the CLI:** https://www.youtube.com/watch?v=EYAnaNnWqBg

## Review Questions

The answers to these review questions appear in Appendix A, "Answers to Review Questions."

1. Consider the following output:

```
C:\> arp -a
Interface: 172.16.202.128 --- 0xb
Internet Address Physical Address Type
172.16.202.2 00-50-56-fd-65-2c dynamic
172.16.202.255 ff-ff-ff-ff-ff-ff static
224.0.0.22 01-00-5e-00-00-16 static
224.0.0.252 01-00-5e-00-00-fc static
255.255.255.255 ff-ff-ff-ff-ff-ff static
```

   In this example, what is the MAC address corresponding to the IP address 172.16.202.2?

   a. ff-ff-ff-ff-ff-ff
   b. 00-50-56-fd-65-2c
   c. 01-00-5e-00-00-16
   d. 01-00-5e-00-00-fc

2. What option would you specify after the **ipconfig** command to display the IP address of a Windows PC's DNS server?

   a. No option is needed because **ipconfig** displays DNS server information by default.
   b. **/full**
   c. **/fqdn**
   d. **/all**

3. What protocol is used by the **ping** command?
   a. IGMP
   b. PIM
   c. ICMP
   d. RTP

4. Which of the following commands is used on a Linux host to generate information about each router hop along the path from a source to a destination?
   a. **ping -t**
   b. **tracert**
   c. **ping -r**
   d. **traceroute**

5. Which of the following Linux commands can be used to check FQDN to IP address resolution? (Choose three.)
   a. **nslookup**
   b. **netstat**
   c. **dig**
   d. **host**

6. What command produced the following snippet of output?

```
OUTPUT OMITTED...
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 62169
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0,
ADDITIONAL: 0
;; QUESTION SECTION:
;pearsonitcertification.com. IN A
;; ANSWER SECTION:
 pearsonitcertification.com. 10800 IN A 64.28.85.25
;; Query time: 202 msec
;; SERVER: 192.168.1.1#53(192.168.1.1)
;; WHEN: Wed Jun 1 20:41:57 2011
;; MSG SIZE rcvd: 60
 OUTPUT OMITTED...
```

a. **traceroute -d pearsonitcertification.com**
b. **dig pearsonitcertification.com**
c. **netstat -a pearsonitcertification.com**
d. **nbtstat pearsonitcertification.com**

7. Which tool would be used as part of a wireless site survey and produces a heat map?
   a. Bandwidth tester
   b. Wi-Fi analyzer
   c. port scanner
   d. iperf

8. What command is used to view the routing table configuration of a network device?
   a. **show config**
   b. **show interface**
   c. **show route**
   d. **show tcpdump**

9. Which of the following are used by devices to identify and share information about themselves within a local area network? (Choose two.)
   a. STP
   b. IDS
   c. LLDP
   d. CDP

*This page intentionally left blank*